

EJEMPLO ABSTRACT FACTORY

Escenario del problema:

Imagina que estamos desarrollando una aplicación que maneja múltiples tipos de interfaces gráficas (GUI) y debe ser compatible tanto con **Windows** como con **macOS**. En nuestro caso, queremos crear botones y cuadros de texto que se ajusten a la apariencia del sistema operativo en el que se ejecuta la aplicación.

✗ Problema inicial (MAL DISEÑADO)

Al principio, podríamos intentar resolver esto con condicionales, como en el siguiente código:

```
class Button {
    void render() {
        System.out.println("Botón genérico");
    }
}

class WindowsButton extends Button {
    void render() {
        System.out.println("Renderizando un botón estilo Windows");
    }
}

class MacOSButton extends Button {
    void render() {
        System.out.println("Renderizando un botón estilo MacOS");
    }
}

class Application {
    private Button button;

    public Application(String osType) {
        if (osType.equals("Windows")) {
            button = new WindowsButton();
        } else if (osType.equals("MacOS")) {
            button = new MacOSButton();
        }
    }

    public void renderUI() {
        button.render();
    }
}
```

¿Por qué este código está MAL?

1. **Alta dependencia de clases concretas:** La clase `Application` tiene lógica condicional (`if-else`) que selecciona qué tipo de `Button` instanciar. Esto viola el principio **Open/Closed (OCP)** porque cada vez que agregamos un nuevo sistema operativo, debemos modificar la clase.
2. **Dificultad para agregar nuevos elementos:** Si queremos agregar más componentes (como `TextBox`, `CheckBox`), tendríamos que replicar la lógica condicional en múltiples lugares.
3. **No sigue el principio de inversión de dependencias:** `Application` depende directamente de las implementaciones concretas (`WindowsButton` y `MacOSButton`).

✓ Solución con Abstract Factory (BIEN DISEÑADO)

Aplicamos el patrón Abstract Factory para desacoplar la creación de los objetos específicos según el sistema operativo.

◆ Paso 1: Definir las interfaces para los productos

Creamos una interfaz base para los botones y otra para los cuadros de texto.

```
// Producto Abstracto: Button
interface Button {
    void render();
}

// Producto Abstracto: TextBox
interface TextBox {
    void render();
}
```

◆ Paso 2: Crear implementaciones concretas para cada sistema operativo

Definimos cómo se verán los botones y cuadros de texto en cada plataforma.

```
// Implementaciones concretas para Windows
class WindowsButton implements Button {
    public void render() {
        System.out.println("Renderizando un botón estilo Windows");
    }
}

class WindowsTextBox implements TextBox {
    public void render() {
        System.out.println("Renderizando un cuadro de texto estilo Windows");
    }
}

// Implementaciones concretas para MacOS
class MacOSButton implements Button {
    public void render() {
        System.out.println("Renderizando un botón estilo MacOS");
    }
}

class MacOSTextBox implements TextBox {
    public void render() {
        System.out.println("Renderizando un cuadro de texto estilo MacOS");
    }
}
```

◆ Paso 3: Definir la fábrica abstracta

Creamos una interfaz para la fábrica de componentes.

```
// Fábrica Abstracta
interface GUIFactory {
    Button createButton();
    TextBox createTextBox();
}
```

◆ Paso 4: Implementar las fábricas concretas

Cada fábrica concreta se encarga de crear los objetos específicos para cada plataforma.

```
// Fábrica concreta para Windows
class WindowsFactory implements GUIFactory {
    public Button createButton() {
        return new WindowsButton();
    }

    public TextBox createTextBox() {
        return new WindowsTextBox();
    }
}

// Fábrica concreta para MacOS
class MacOSFactory implements GUIFactory {
    public Button createButton() {
        return new MacOSButton();
    }

    public TextBox createTextBox() {
        return new MacOSTextBox();
    }
}
```

♦ Paso 5: Utilizar la fábrica en la aplicación

Nuestra aplicación ahora recibe la fábrica adecuada y usa los productos sin conocer sus detalles concretos.

♦ Paso 5: Utilizar la fábrica en la aplicación

Nuestra aplicación ahora recibe la fábrica adecuada y usa los productos sin conocer sus detalles concretos.

```
class Application {
    private Button button;
    private TextBox textBox;

    public Application(GUIFactory factory) {
        this.button = factory.createButton();
        this.textBox = factory.createTextBox();
    }

    public void renderUI() {
        button.render();
        textBox.render();
    }
}
```

♦ Paso 6: Crear la aplicación en base al sistema operativo

En la función principal, decidimos qué fábrica usar según el sistema operativo.

```
public class Main {
    public static void main(String[] args) {
        GUIFactory factory;
        String osType = System.getProperty("os.name"); // Obtiene el OS del sistema

        if (osType.contains("Windows")) {
            factory = new WindowsFactory();
        } else {
            factory = new MacOSFactory();
        }

        Application app = new Application(factory);
        app.renderUI();
    }
}
```

✗ Problema Inicial (MAL DISEÑADO)

Escenario del Problema

Imaginemos que estamos diseñando una aplicación que maneja diferentes tipos de notificaciones para los usuarios:

- Correo Electrónico (EmailNotification)
- Mensaje SMS (SMSNotification)
- Notificación Push (PushNotification)

Inicialmente, podríamos implementar esto usando condicionales para crear los objetos de notificación en la clase principal:

```
class EmailNotification {
    void notifyUser() {
        System.out.println("Enviando notificación por correo electrónico...");
    }
}

class SMSNotification {
    void notifyUser() {
        System.out.println("Enviando notificación por SMS...");
    }
}

class PushNotification {
    void notifyUser() {
        System.out.println("Enviando notificación push...");
    }
}

// Clase que gestiona el envío de notificaciones
class NotificationService {
    private String notificationType;

    public NotificationService(String notificationType) {
        this.notificationType = notificationType;
    }

    public void sendNotification() {
        if (notificationType.equals("Email")) {
            EmailNotification email = new EmailNotification();
            email.notifyUser();
        } else if (notificationType.equals("SMS")) {
            SMSNotification sms = new SMSNotification();
            sms.notifyUser();
        } else if (notificationType.equals("Push")) {
            PushNotification push = new PushNotification();
            push.notifyUser();
        } else {
            System.out.println("Tipo de notificación no soportado.");
        }
    }
}
```

¿Por qué este código está MAL?

1. Violación del principio Open/Closed (OCP): Si queremos agregar un nuevo tipo de notificación, tenemos que modificar la clase `NotificationService`, lo que no es escalable.
2. Alta dependencia de clases concretas: `NotificationService` depende directamente de `EmailNotification`, `SMSNotification` y `PushNotification`, lo que genera acoplamiento.
3. Dificultad en la extensión: Si queremos agregar notificaciones WhatsApp, Telegram, etc., debemos modificar la lógica condicional.

✓ Solución con el Patrón Factory Method (BIEN DISEÑADO)

Aplicamos Factory Method para eliminar los condicionales y mejorar la extensibilidad.

◆ Paso 1: Crear una interfaz para las notificaciones

```
// Interfaz común para todas las notificaciones
interface Notification {
    void notifyUser();
}
```

◆ Paso 2: Implementaciones concretas para cada tipo de notificación

```
class EmailNotification implements Notification {
    public void notifyUser() {
        System.out.println("Enviando notificación por correo electrónico...");
    }
}

class SMSNotification implements Notification {
    public void notifyUser() {
        System.out.println("Enviando notificación por SMS...");
    }
}

class PushNotification implements Notification {
    public void notifyUser() {
        System.out.println("Enviando notificación push...");
    }
}
```

♦ Paso 3: Definir la fábrica abstracta (Factory Method)

Creamos una clase base que define el método `createNotification()`, que será sobrescrito por cada subclase.

```
// Fábrica abstracta con el Factory Method
abstract class NotificationFactory {
    public abstract Notification createNotification();
}
```

♦ Paso 4: Implementar las fábricas concretas

Cada fábrica se encarga de crear su propio tipo de notificación.

Cada fábrica se encarga de crear su propio tipo de notificación.

```
java Copy Edit

class EmailNotificationFactory extends NotificationFactory {
    public Notification createNotification() {
        return new EmailNotification();
    }
}

class SMSNotificationFactory extends NotificationFactory {
    public Notification createNotification() {
        return new SMSNotification();
    }
}

class PushNotificationFactory extends NotificationFactory {
    public Notification createNotification() {
        return new PushNotification();
    }
}
```

♦ Paso 5: Usar la fábrica en la aplicación

Ahora, en lugar de depender de clases concretas dentro de `NotificationService`, usamos la fábrica para crear notificaciones dinámicamente.

```
class NotificationService {
    private NotificationFactory factory;

    public NotificationService(NotificationFactory factory) {
        this.factory = factory;
    }

    public void sendNotification() {
        Notification notification = factory.createNotification();
        notification.notifyUser();
    }
}
```


♦ Paso 6: Crear y usar la aplicación con diferentes notificaciones

Ahora la creación de notificaciones está totalmente desacoplada.

```
public class Main {  
    public static void main(String[] args) {  
        NotificationFactory factory;  
  
        // Simulación de entrada dinámica (por ejemplo, desde una configuración)  
        String notificationType = "SMS"; // Puede ser "Email", "Push", "SMS"  
  
        if (notificationType.equals("Email")) {  
            factory = new EmailNotificationFactory();  
        } else if (notificationType.equals("SMS")) {  
            factory = new SMSNotificationFactory();  
        } else {  
            factory = new PushNotificationFactory();  
        }  
  
        // Crear el servicio con la fábrica correcta  
        NotificationService service = new NotificationService(factory);  
        service.sendNotification();  
    }  
}
```

DIFERENCIAS

Ejemplo 1: Factory Method (🏭)

- 💡 Caso: Crear un solo producto específico (notificaciones).
- 💡 Cada tipo de notificación tiene su propia fábrica.
- 💡 Cada fábrica crea SOLO un tipo de objeto (EmailNotification, SMSNotification, etc.).

```
java                                                                    Copy Edit  
  
abstract class NotificationFactory {  
    public abstract Notification createNotification();  
}  
  
// Cada fábrica concreta fabrica SOLO UN tipo de notificación  
class EmailNotificationFactory extends NotificationFactory {  
    public Notification createNotification() {  
        return new EmailNotification();  
    }  
}  
  
class SMSNotificationFactory extends NotificationFactory {  
    public Notification createNotification() {  
        return new SMSNotification();  
    }  
}  
  
// Se usa la fábrica para crear objetos sin depender de clases concretas  
NotificationFactory factory = new EmailNotificationFactory();  
Notification notification = factory.createNotification();  
notification.notifyUser();
```

- ✅ Aquí cada fábrica solo crea UN tipo de producto, por eso es un Factory Method.

Ejemplo 2: Abstract Factory (GUI)

- 💡 Caso: Crear varios productos relacionados (botón y cuadro de texto).
- 💡 Una sola fábrica se encarga de crear una familia de productos.
- 💡 Ejemplo: WindowsFactory crea un botón y un cuadro de texto estilo Windows.

```
java                                                                    Copy Edit

// Interfaz para la fábrica
interface GUIFactory {
    Button createButton();
    TextBox createTextBox();
}

// Fábrica concreta que crea MÚLTIPLES objetos relacionados (Windows)
class WindowsFactory implements GUIFactory {
    public Button createButton() {
        return new WindowsButton();
    }
    public TextBox createTextBox() {
        return new WindowsTextBox();
    }
}

// Se usa la fábrica para obtener distintos objetos relacionados
GUIFactory factory = new WindowsFactory();
Button btn = factory.createButton();
TextBox txt = factory.createTextBox();
btn.render();
txt.render();
```

- ✅ Aquí una única fábrica crea múltiples productos (botón y cuadro de texto) de una misma familia, por eso es un Abstract Factory.

EJEMPLO BUILD

✗ Problema Inicial (MAL DISEÑADO)

Escenario del Problema

Imagina que estamos desarrollando un sistema para generar perfiles de usuario en una aplicación. Un perfil de usuario tiene múltiples atributos opcionales, como:

- Nombre (obligatorio)
- Correo electrónico (opcional)
- Edad (opcional)
- Dirección (opcional)
- Número de teléfono (opcional)

Inicialmente, podríamos intentar resolverlo usando constructores con múltiples parámetros.


```

class User {
    private String name;
    private String email;
    private int age;
    private String address;
    private String phoneNumber;

    public User(String name, String email, int age, String address, String phoneNumber) {
        this.name = name;
        this.email = email;
        this.age = age;
        this.address = address;
        this.phoneNumber = phoneNumber;
    }

    public void display() {
        System.out.println("Nombre: " + name);
        System.out.println("Correo: " + email);
        System.out.println("Edad: " + age);
        System.out.println("Dirección: " + address);
        System.out.println("Teléfono: " + phoneNumber);
    }
}

```

¿Por qué este código está MAL?

1. Constructores difíciles de manejar: Si hay muchos atributos opcionales, terminamos con varios constructores sobrecargados.
2. Orden estricto de los parámetros: Si pasamos `null` en el lugar incorrecto, obtenemos errores inesperados.
3. Falta de flexibilidad: No podemos crear el objeto de manera progresiva, sino que debemos proporcionar todos los valores de inmediato.

✓ Solución con el Patrón Builder (BIEN DISEÑADO)

¿Cuándo usar Builder?

- ✓ Cuando queremos construir un objeto paso a paso.
- ✓ Cuando hay múltiples atributos opcionales y no queremos usar un constructor gigante.
- ✓ Cuando queremos usar un objeto de manera parcial antes de completarlo.

◆ Paso 1: Definir la clase `User`

El constructor es privado, porque usaremos `Builder` para crear instancias.

```

class User {
    private String name;
    private String email;
    private int age;
    private String address;
    private String phoneNumber;

    private User(UserBuilder builder) {
        this.name = builder.name;
        this.email = builder.email;
        this.age = builder.age;
        this.address = builder.address;
        this.phoneNumber = builder.phoneNumber;
    }

    public void display() {
        System.out.println("Nombre: " + name);
        System.out.println("Correo: " + (email != null ? email : "No especificado"));
        System.out.println("Edad: " + (age != 0 ? age : "No especificado"));
        System.out.println("Dirección: " + (address != null ? address : "No especificado"));
        System.out.println("Teléfono: " + (phoneNumber != null ? phoneNumber : "No especificado"));
    }

    // Método para usar Builder
    public static UserBuilder builder(String name) {
        return new UserBuilder(name);
    }
}

```

♦ Paso 2: Implementar el Builder

El **Builder** nos permite construir el objeto paso a paso.

```

class UserBuilder {
    String name;
    String email;
    int age;
    String address;
    String phoneNumber;

    public UserBuilder(String name) {
        this.name = name; // Nombre es obligatorio
    }

    public UserBuilder withEmail(String email) {
        this.email = email;
        return this;
    }

    public UserBuilder withAge(int age) {
        this.age = age;
        return this;
    }

    public UserBuilder withAddress(String address) {
        this.address = address;
        return this;
    }

    public UserBuilder withPhoneNumber(String phoneNumber) {
        this.phoneNumber = phoneNumber;
        return this;
    }

    public User build() {
        return new User(this);
    }
}

```

♦ Paso 3: Usar el Builder para crear objetos progresivamente

Ahora podemos construir el usuario poco a poco, agregando solo los atributos necesarios.

```
public class Main {
    public static void main(String[] args) {
        // Se crea un usuario solo con el nombre
        User user1 = User.builder("David")
            .withEmail("david@example.com")
            .build();

        // Se crea un usuario con nombre, edad y dirección
        User user2 = User.builder("Carmen")
            .withAge(30)
            .withAddress("Madrid, España")
            .build();

        // Se crea un usuario completo
        User user3 = User.builder("María")
            .withEmail("maria@example.com")
            .withAge(28)
            .withAddress("Barcelona, España")
            .withPhoneNumber("600123456")
            .build();

        // Mostrar los datos
        System.out.println("Usuario 1:");
        user1.display();
        System.out.println("\nUsuario 2:");
        user2.display();
        System.out.println("\nUsuario 3:");
        user3.display();
    }
}
```

EJEMPLO PROTOTYPE

✗ Problema Inicial (MAL DISEÑADO)

Escenario del Problema

Imaginemos que estamos desarrollando un sistema de gestión de documentos, donde cada documento puede contener diferentes tipos de contenido (texto, imágenes, tablas, etc.).

💡 Problema: Crear copias de documentos puede ser costoso en términos de rendimiento si instanciamos nuevos objetos desde cero y copiamos sus datos uno por uno.

♦ Versión con Creación Manual (MAL DISEÑADA)

Cada vez que necesitamos duplicar un documento, lo creamos desde cero y copiamos sus atributos manualmente.

```
class Document {
    private String title;
    private String content;

    public Document(String title, String content) {
        this.title = title;
        this.content = content;
    }

    public void display() {
        System.out.println("Título: " + title);
        System.out.println("Contenido: " + content);
    }

    // Método manual para copiar el objeto
    public Document copy() {
        return new Document(this.title, this.content);
    }
}
```

Uso de la clase:

```

public class Main {
    public static void main(String[] args) {
        // Crear un documento original
        Document doc1 = new Document("Contrato", "Este es un contrato de servicios...");

        // Copiar el documento de forma manual
        Document doc2 = doc1.copy();

        doc1.display();
        System.out.println("-----");
        doc2.display();
    }
}

```

¿Por qué este código está MAL?

1. Duplicación manual: Si la clase `Document` tiene muchos atributos, debemos copiarlos manualmente en el constructor.
2. No es flexible: Si agregamos nuevos atributos, debemos modificar el método `copy()`.
3. Alto costo de creación: Si `Document` fuera más complejo, la copia manual sería ineficiente.

✓ Solución con el Patrón Prototype (BIEN DISEÑADO)

💡 ¿Cuándo usar Prototype?

- ✓ Cuando queremos clonar objetos de forma eficiente, sin depender de constructores costosos.
- ✓ Cuando la creación de objetos es cara o complicada y queremos evitar instancias repetitivas.
- ✓ Cuando queremos copiar objetos parcialmente modificados sin alterar el original.

♦ Paso 1: Definir una interfaz Prototype

Creamos una interfaz que obliga a las clases a implementar un método `clone()`.

```

interface Prototype {
    Prototype clone();
}

```

♦ Paso 2: Implementar la clase `Document` con Prototype

Ahora `Document` implementa `Prototype`, permitiendo la clonación automática.

```

class Document implements Prototype {
    private String title;
    private String content;

    public Document(String title, String content) {
        this.title = title;
        this.content = content;
    }

    // Implementación del método clone()
    @Override
    public Prototype clone() {
        return new Document(this.title, this.content);
    }

    public void display() {
        System.out.println("Título: " + title);
        System.out.println("Contenido: " + content);
    }
}

```

♦ Paso 3: Uso del Prototype en la Aplicación

Ahora podemos clonar objetos sin necesidad de instanciarlos manualmente.

```

public class Main {
    public static void main(String[] args) {
        // Crear un documento original
        Document doc1 = new Document("Contrato", "Este es un contrato de servicios...");

        // Clonar el documento usando el patrón Prototype
        Document doc2 = (Document) doc1.clone();

        // Mostrar ambos documentos
        doc1.display();
        System.out.println("-----");
        doc2.display();
    }
}

```

Beneficios del Patrón Prototype

- ✓ Evita la sobrecarga de creación: En lugar de construir objetos desde cero, los clonamos.
- ✓ Fácil de mantener: No necesitamos actualizar manualmente un método `copy()` si se agregan nuevos atributos.
- ✓ Flexibilidad: Se pueden clonar objetos y luego modificar la copia sin afectar el original.

Sin usar PROTOTYPE, cada vez que añadimos un nuevo atributo a la clase, tenemos que modificar el método copy().

Usando PROTOTYPE, cada vez que añadimos un nuevo atributo a la clase, tenemos que modificar el método clone().

¿Cuál es la ventaja entonces de usar este patrón?

1 Problema con una copia manual (sin Prototype)

Si no usamos Prototype, cada vez que añadimos un nuevo atributo a la clase, debemos recordar modificar todas las copias manuales que hemos hecho en distintas partes del código.

Ejemplo sin Prototype:

```
class Document {
    private String title;
    private String content;
    private String author; // Nuevo atributo

    public Document(String title, String content, String author) {
        this.title = title;
        this.content = content;
        this.author = author;
    }

    public Document copy() {
        // Ahora debemos recordar copiar el nuevo atributo
        return new Document(this.title, this.content, this.author);
    }
}
```

Si olvidamos modificar `copy()`, el nuevo atributo `author` no se copiará, lo que podría generar errores difíciles de detectar.

2 Cómo Prototype mejora esto

En el Patrón Prototype, la responsabilidad de la clonación se centraliza en una interfaz común (Prototype).

Esto asegura que todas las clases que implementen `Prototype` deben proporcionar una implementación de `clone()`, lo que facilita el mantenimiento.

```
interface Prototype {
    Prototype clone();
}

class Document implements Prototype {
    private String title;
    private String content;
    private String author; // Nuevo atributo

    public Document(String title, String content, String author) {
        this.title = title;
        this.content = content;
        this.author = author;
    }

    @Override
    public Prototype clone() {
        return new Document(this.title, this.content, this.author); // Clonación automática
    }
}
```


¿Por qué esto es mejor?

- ✓ Centralización: Todos los objetos que implementan `Prototype` tienen un mecanismo uniforme de clonación.
- ✓ Evita errores en múltiples partes del código: Si olvidamos modificar `clone()`, el compilador nos obliga a implementarlo correctamente.
- ✓ Permite polimorfismo: Podemos clonar diferentes tipos de objetos sin depender de sus implementaciones concretas.

POLIMORFISMO

Supongamos que tenemos una interfaz `Animal` con un método `makeSound()`, que cada animal implementará de manera diferente.

```
// Definimos la interfaz Animal
interface Animal {
    void makeSound();
}

// Implementaciones concretas
class Dog implements Animal {
    public void makeSound() {
        System.out.println("Guau Guau!");
    }
}

class Cat implements Animal {
    public void makeSound() {
        System.out.println("Miau Miau!");
    }
}

class Cow implements Animal {
    public void makeSound() {
        System.out.println("Muuuuu!");
    }
}
```

Uso del Polimorfismo en el código

En lugar de hacer `if-else` para cada tipo de animal, podemos usar polimorfismo.

```
public class Main {
    public static void main(String[] args) {
        // Creamos un array de animales (polimorfismo en acción)
        Animal[] animals = {new Dog(), new Cat(), new Cow()};

        // Llamamos al método makeSound() sin preocuparnos del tipo específico de objeto
        for (Animal animal : animals) {
            animal.makeSound();
        }
    }
}
```

🔴 ¿Se puede usar polimorfismo sin Prototype?

Sí, y de hecho, lo usamos constantemente en herencia, interfaces y métodos sobrescritos, como en el ejemplo de `Animal` que te mostré antes.

Ejemplo sin Prototype:

```
// Interfaz común
interface Shape {
    void draw();
}

// Implementaciones concretas
class Circle implements Shape {
    public void draw() {
        System.out.println("Dibujando un círculo");
    }
}

class Square implements Shape {
    public void draw() {
        System.out.println("Dibujando un cuadrado");
    }
}

public class Main {
    public static void main(String[] args) {
        // Polimorfismo en acción sin Prototype
        Shape shape1 = new Circle();
        Shape shape2 = new Square();

        shape1.draw(); // Dibujando un círculo
        shape2.draw(); // Dibujando un cuadrado
    }
}
```

🔴 ¿Qué aporta Prototype al Polimorfismo?

Sin Prototype, no podemos clonar objetos polimórficos sin conocer su tipo concreto.

Ejemplo sin Prototype (NO se puede clonar polimórficamente):

```
Shape shape = new Circle();
Shape clonedShape = new Shape(shape); // ❌ Esto no funciona, porque Java no sabe cómo clonar
```

Con Prototype, sí podemos clonar polimórficamente:

```
interface Prototype {
    Prototype clone();
}

class Circle implements Prototype {
    public Prototype clone() {
        return new Circle();
    }
}

class Square implements Prototype {
    public Prototype clone() {
        return new Square();
    }
}

public class Main {
    public static void main(String[] args) {
        Prototype shape = new Circle(); // Polimorfismo
        Prototype clonedShape = shape.clone(); // ✅ Clonación polimórfica
    }
}
```

1 Copia Manual con `copy()` (Sin Prototype)

Aquí, si queremos clonar objetos polimórficamente, NO podemos hacerlo sin usar `instanceof` o `if-else`.

```
java                                                                    Copy Edit

abstract class Shape {
    String color;

    public Shape(String color) {
        this.color = color;
    }

    public abstract Shape copy();
}

class Circle extends Shape {
    int radius;

    public Circle(String color, int radius) {
        super(color);
        this.radius = radius;
    }

    @Override
    public Circle copy() {
        return new Circle(this.color, this.radius);
    }
}

class Square extends Shape {
    int sideLength;

    public Square(String color, int sideLength) {
        super(color);
        this.sideLength = sideLength;
    }

    @Override
    public Square copy() {
        return new Square(this.color, this.sideLength);
    }
}

public class Main {
    public static void main(String[] args) {
        Shape shape = new Circle("Rojo", 10);

        // 🚩 Si queremos copiarlo, DEBEMOS conocer su tipo
        Shape copiedShape;
        if (shape instanceof Circle) {
            copiedShape = ((Circle) shape).copy();
        } else if (shape instanceof Square) {
            copiedShape = ((Square) shape).copy();
        } else {
            copiedShape = null;
        }

        System.out.println("Copia exitosa: " + (copiedShape != null));
    }
}
```

● Problema con `copy()`

- Dependemos del tipo exacto (`instanceof`, `if-else`).
- Si agregamos `Triangle`, debemos modificar TODO el código donde copiamos objetos.

2 Clonación con Prototype

Aquí, ya no necesitamos saber el tipo exacto para clonar objetos.

```
java Copy Edit

interface Prototype {
    Prototype clone();
}

abstract class Shape implements Prototype {
    String color;

    public Shape(String color) {
        this.color = color;
    }
}

class Circle extends Shape {
    int radius;

    public Circle(String color, int radius) {
        super(color);
        this.radius = radius;
    }

    @Override
    public Circle clone() {
        return new Circle(this.color, this.radius);
    }
}
```

```
class Square extends Shape {
    int sideLength;

    public Square(String color, int sideLength) {
        super(color);
        this.sideLength = sideLength;
    }

    @Override
    public Square clone() {
        return new Square(this.color, this.sideLength);
    }
}

public class Main {
    public static void main(String[] args) {
        Shape shape = new Circle("Rojo", 10);

        // ✅ Podemos clonar sin importar qué tipo de objeto es
        Shape copiedShape = shape.clone();

        System.out.println("Copia exitosa: " + (copiedShape != null));
    }
}
```

● Ventaja con Prototype

- No dependemos del tipo exacto.
- No necesitamos `instanceof` ni `if-else` → el polimorfismo se encarga.
- Si agregamos `Triangle`, NO necesitamos modificar `Main`.